Now: We get $V(s)$ $Q(s,a)$ By a table (tabular method)
storing all V, Q

Real-World Problem: Too many States! Cannot Store in a Table

VFA: Value Function Approximation

Approximators: (linear combinations of features
NN, Decision Tree, Nearest Neighbor ···

- Q: Which function approximation should you choose?
- This depends on your goals:
  - (standard) Tabular: good theory but does not scale/generalize
  - Linear: reasonably good theory, but requires good features
  - Non-linear: less well-understood, but scales well
  - Flexible, and less reliant on picking good features first (*e.g.*, by hand)
  - (Deep) neural nets often perform quite well, and remain a popular choice
    - This is often called deep reinforcement learning, when using (deep) neural network to represent these functions

Linear Function Approximation

$S_t = O_t$ Fixed feature map: $x: S \to \mathbb{R}^n$

Values are linear function of features $V(s;w) = w^T x(s)$

$S \to x(s) \to V(s,w)$

Solution for Large MDPs

$V(s;w) \approx V_\pi(s)$ $Q(s,a;w) \approx Q_\pi(s,a)$

Seen States $\xrightarrow{\text{generalize}}$ unseen State

Update w: MC/TD

Differentiable function Approximation

$V(s;w)$: Differentiable, may not linear

RL: { data not i.i.d.
policy affects data received
regression targets can be non-stationary

VFA

$S_t \to (w) \to V(S_t;w)$

$S_t, A_t \to (w) \to Q(S_t, A_t, w)$

Updating w $\longleftrightarrow$ V, Q may change!

Policy Approximation

$S_t \to [w] \to \hat{\pi}(A_t \mid S_t, w)$

Function Approximation

classification: feature $x \to f(x)$: class score

CNN / RNN $h_t = f_w(h_{t-1}, x_t)$

Agent State Update

When not fully observable ENV: $S_t^{env} \neq O_t$.

$S_t = U(S_{t-1}, a_{t-1}, o_t; w)$

↖ Agent State

Gradient-Based Algorithm

$J(w)$: Differentiable function

Gradient: $\nabla J(w) = \begin{pmatrix} \frac{\partial J(w)}{\partial w_1} \\ \vdots \\ \frac{\partial J(w)}{\partial w_n} \end{pmatrix}$

Gradient Descent $\Delta w = \frac{1}{2}\alpha(\nabla_w J(w))$ (Direction: $-\nabla J$; step size)

$J(w_1) \xrightarrow{\Delta w} J(w_2) \xrightarrow{\Delta w} \cdots$

$J(w_1) \geq J(w_2) \cdots \geq J(w_n)$

VFA By Stochastic GD

$J(w) = E_{s\sim d}[(V_\pi(s) - V(s;w))^2]$

d: distribution over States (induced by policy / dynamics)

$\nabla_w J(w) = -2 E_d[(V_\pi(s) - V(s;w))\nabla_w V(s;w)]$

$\Delta w = -\frac{1}{2}\alpha \cdot \nabla_w J(w) = \alpha \nabla_w J(w) = \alpha E_d[(V_\pi(s) - V(s;w))\nabla_w V(s;w)]$

Feature Vectors $x(s) = \begin{pmatrix} x_1(s) \\ \vdots \\ x_n(s) \end{pmatrix}$ $x: S \to \mathbb{R}^n$ mapping state to features

(Store)

Shore hand: $x_t = x(s_t)$

Linear Value Function Approximation

$V(s;w) = x(s)^T w = \sum_{j=1}^n x_j(s) w_j$

$J(w) = E_{s\sim d}[(V_\pi(s) - x(s)^T w)^2]$

$\nabla_w V(s;w) = \begin{pmatrix} x_1(s) \\ x_2(s) \\ \vdots \\ x_n(s) \end{pmatrix} = x(s_t) = x_t$

$\Delta w = \alpha(V_\pi(s_t) - V(s_t;w))x_t$

Linear
Update = Step-size × prediction error × feature value

$V_\pi(s)$: In RL, we can't know it

⇒ Substitute a target for $V_\pi(s)$

MC: $\Delta w = \alpha(G_t - V(s_t;w))\nabla_w V(s_t;w)$

TD(0): $\Delta w = \alpha(r_{t+1} + \gamma V(s_{t+1};w) - V(s_t;w))\nabla_w V(s_t;w)$

TD(λ): $\Delta w = \alpha(G_t^\lambda - V(s_t;w))\nabla_w V(s_t;w)$

$G_{t:t}^\lambda = r_t + \gamma((1-\lambda)V(s_{t+1};w) + \lambda G_{t+1}^\lambda)$

MC . VFA

G_t: unbiased, noisy Sample of $V_\pi(s)$

$\langle S_1, G_1\rangle, \langle S_2, G_2\rangle, \cdots, \langle S_t, G_T\rangle$: data ⇒ Supervised learning!

Linear MC VFA (Linear regression)

$\Delta w = \alpha \cdot (G_t - V(s_t;w)) \cdot x_t$ ⇒ Converge to Local minimum
$= w^T x_t$

non-Linear also Converge to Local MIN

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v}: \mathbb{S} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using π
  Loop for each step of episode, $t = 0, 1, \ldots, T-1$:
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha[G_t - \hat{v}(S_t,\mathbf{w})]\nabla\hat{v}(S_t,\mathbf{w})$

Actually Supervised learning

TD. VFA

target: $r_{t+1} + \gamma V(s_{t+1};w)$ biased

Still can be used as training data ⇒ Supervised learning

TD(0) with linear VFA

$\Delta w = \alpha(r_t + \gamma V(s_{t+1};w) - V(s_t;w))\nabla_w V(s_t;w)$

$= \alpha(r_t + \gamma V(s_{t+1};w) - V(s_t;w))x_t$

TD Error

We want: $V(s_t;w)$ close $\to$ $r_t + \gamma V(s_{t+1};w)$

But, when updating w, the target change!

Non-stationary regression

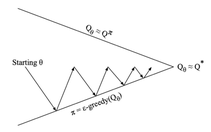
Semi-gradient method

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v}: \mathbb{S}^+ \times \mathbb{R}^d \to \mathbb{R}$ such that $\hat{v}(\text{terminal},\cdot) = 0$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose $A \sim \pi(\cdot|S)$
    Take action A, observe R, S'
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha[R + \gamma\hat{v}(S',\mathbf{w}) - \hat{v}(S,\mathbf{w})]\nabla\hat{v}(S,\mathbf{w})$
    $S \leftarrow S'$
  until S is terminal

Control with VFA



- **Policy evaluation**: Approximate policy evaluation, $Q(s_t, a_t; \boldsymbol{w}) \approx Q_\pi$
- **Policy improvement**: ε-greedy policy improvement

Action in / Action out

Action in: $Q(s,a;w) = w^T x(s,a)$
Action out: $Q(s;w) = w^T x(s)$

⇒ { continuous action ⇒ use action-in. (e.g. rotate π^*)
{ discrete action ⇒ action out (e.g. up/down/left)

Convergence / Divergence

Convergence of MC VFA (Linear)

$w_{mc} = \arg\min_w E_\pi[(G_t - V_w(s_t))^2] = E_\pi[x_t x_t^T]^{-1} E_\pi[G_t x_t]$

Converge

proof:

When Stopping: $\nabla_w = 0$

$\Rightarrow \nabla_w E[(G_t - V_w(s_t))^2] = -2E[(G_t - V_w(s_t))x_t] = 0$.

$\Rightarrow E[(G_t - V_w(s_t))x_t] = 0$

$E[G_t x_t - x_t x_t^T w_{MC}] = 0$.

$w_{MC} = E[x_t x_t^T]^{-1} E[G_t x_t]$

Convergence of TD (linear VFA)

$w_{TD} = E[x_t(x_t - \gamma x_{t+1})^T]^{-1} E[r_t x_t]$

proof:

$E[\Delta w_{TD}] = 0 = E[\alpha_t(r_t + \gamma x_{t+1}^T w_{TD} - x_t^T w_{TD})x_t]$

$0 = E[(r_t + \gamma x_{t+1}^T w_{TD} - x_t^T w_{TD})x_t]$ (Assuming α_t doesn't Correlate to r, x)

$w_{TD} = E[x_t(x_t^T - x_{t+1}^T)]^{-1} E[r_t x_t]$

{ MC: smaller prediction error
{ TD: converge faster

$\overline{VE}(w)$: value error

$\overline{VE}(w) = \|V_\pi - V_w\|_{d_\pi}^2 = \sum_{s\in S} d_\pi(s)(V_\pi(s) - V_w(s))^2$.

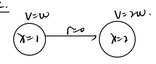
distribution over policy π

MC minimize value error

$\overline{VE}(w_{TD}) \leq \frac{1}{1-\gamma}\overline{VE}(w_{mc}) = \frac{1}{1-\gamma}\min_w \overline{VE}(w)$.

Divergence.

Example



$w_{t+1} = w_t - \alpha_t(r + \gamma V(s') - V(s))\nabla V(s)$.

$= w_t - \alpha_t(r + \gamma V(s') - V(s))x(s)$

$= w_t + \alpha_t(0 + \gamma 2w - w)\cdot 1$

$= w_t + \alpha_t(2\gamma - 1)w_t$.

If $2\gamma > 1$, $\gamma > \frac{1}{2}$ then $w_{t+1} > w_t$. $\lim_{t\to\infty} w_t = \infty$ ⇒ Divergent!

第二种情况下，人为截断和重置改变了训练数据的状态分布，数据不再覆盖原任务按目标策略正常运行时的分布，所以不能直接套用标准 ~~on-policy 线性~~ TD 的收敛保证。也不能仅凭"漏掉第二段"就断言动作策略是 off-policy。

We keep training TD only on this transition

Alg that Combines: Bootstrapping, Off-policy learning, VFA may diverge

⇒ Deadly Triad (① Bootstrapping ② Off-policy ③ VFA)

On/Off-Policy	Algorithm	Table Lookup	Linear	Non-Linear
On-Policy	MC	✓	✓	✓
	TD	✓	✓	×
Off-Policy	MC	✓	✓	✓
	TD	✓	×	×

May Diverge
Although is not deadly triad

Actually, Tracking is more preferred to Convergence

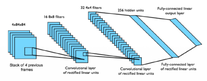
adapting policy over time ← find optimal

Since: environment rarely stay Static → Algorithm need to maintain plasticity (always can learn/change)

Deep Q Network

DQN in Atari

- End-to-end learning of values $Q(s,a)$ from pixels s
- Input state s is a stack of raw pixels from the last 4 frames
- Output is $Q(s,a)$ for 18 joystick/button positions
- Reward is the *change in score* for the step



$Q(s,a)$: learn by DL network

$S \to$ DQN $\to Q(s,a)$

- TD error: $\delta_t = Q_t - (y_t)$ where $Q_t = Q(s_t, a_t; \boldsymbol{w})$ (y_t: $r_t + \gamma \max_a Q(s_{t+1}, a; w)$ (Q-learning Alg))
- TD learning: Find $\boldsymbol{w}$ by minimizing $L(\boldsymbol{w}) = \frac{1}{T}\sum_{t=1}^T \frac{\delta_t^2}{2}$
- Online gradient descent:
  - Observe (s_t, a_t, r_t, s_{t+1}) and compute δ_t
  - Compute gradient $\boldsymbol{g}_t = \frac{\partial \delta_t^2/2}{\partial w} = \delta_t \cdot \frac{\partial Q(s_t,a_t;w)}{\partial w}$ Gradient Descent
  - Gradient descent: $\boldsymbol{w} \leftarrow \boldsymbol{w} - \alpha \cdot \boldsymbol{g}_t$
  - Discard (s_t, a_t, r_t, s_{t+1}) after using it

Waste of experience

We store all experience → We randomly select exp for learning

if we use it Sequentially: data are Strong related ✗ DL: want data to be i.i.d